

پرسش ۱ (۲ نمره) - مشخص کنید هر یک از گزینه‌های زیر درست یا نادرست هستند. توضیح لازم نیست.
(مجرد = abstract)، (مجازی خالص = pure virtual)

(الف) اگر یک کلاس متدی را مجازی خالص تعریف کند، زیرکلاس‌هایش باید آن متد را پیاده‌سازی کنند تا مجرد محسوب نشوند.

درست

(ب) تمام متدهای مجازی در یک کلاس مجرد باید مجازی خالص تعریف شوند. **نادرست**

(ج) تعریف شیء از یک کلاس مجرد موجب خطای زمان کامپایل می‌شود. **درست**

(د) اشاره‌کردن به یک شیء از نوع یک زیرکلاس با اشاره‌گری از نوع سوپرکلاس خطرناک است. **نادرست**

پرسش ۲ (۴ نمره) - در کد زیر ۴ غلط کامپایل را تشخیص دهید و مختصراً توضیح دهید.

<pre>#include <iostream> #include <string> #include <vector> using namespace std; class B { public: B(string t, int a) { title = t; age = a; } void grow(int n); bool olderThan(B b) { return age > b.age; } private: string title; int age; };</pre>	<pre>class A { public: A(string n, string t, int a) : theB(t, a) { theB = B(t, a); name = n; } private: string name; B theB; }; void B::grow(int n) { theB.age += n; } int main() { vector<A> v(100); // A::A() not defined v.push_back(newA("Gholam", "Boss", 50)); return 0; }</pre>
---	--

پرسش ۳ (۴ نمره) – فرض کنید کلاس Frac اعداد کسری را مدل می‌کند. اگر قطعه کد زیر قابل کامپایل باشد بخش عمومی (public) از کلاس Frac را بنویسید. نیازی به نوشتن بخش خصوصی و بدنه متدها نیست.

```
string s;
Frac f(1, 2);
f.add(Frac(2));
if (f.equals(Frac(10, 4)))
    s = f.toString();
class Frac {
public:
    Frac(int n, int d);
    Frac(int n); // or default value for d in the other constructor
    void add(Frac f); // or const Frac&
    bool equals(Frac f); // or const Frac&
    string toString();
private:
    // ...
};
```

پرسش ۴ (۴ نمره) – با توجه به سه کلاس تعریف شده در زیر، نتیجه کامپایل/اجرای هر یک از قطعه کدهای (الف) تا (د) را مشخص کنید.

<pre>class Base { public: virtual void f() = 0; void g() { cout << "Base::g" << endl; } }; class Vase : public Base { public: void g() { cout<< "Vase::g" << endl; } void h() { k(); } void k() { cout << "Vase::k" << endl; } }; class Case : public Vase { public: void f() { cout << "Case::f" << endl; } void k() { cout << "Case::k" << endl; } };</pre>	(الف) <pre>Case c; Vase* vp = &c; vp->k(); Vase::k</pre> (ب) <pre>Case c; Vase* vp = &c; vp->h(); Vase::k</pre> (ج) <pre>Case c; Base* bp = &c; bp->g(); Base::g</pre> (د) <pre>Vase v; // compile error Base* bp = &v; bp->g();</pre>
---	--

پرسش ۵ (۶ نمره) – در یک شرکت دو نوع کارمند مشغول هستند. نوع اول، حقوق ساعتی دریافت می‌کنند. این دسته، نرخ دستمزد ثابتی برای هر ساعت دارند و حقوق آنها حاصل ضرب این نرخ در تعداد ساعت کار آنها می‌باشد. نوع دوم، حقوق ثابت دارند، به این ترتیب که برای ۲۰۰ ساعت کارکرد خود در ماه، مبلغ ثابتی دریافت می‌کنند و برای بیشتر از ۲۰۰ ساعت، به ازای هر ساعت ۵۰٪ اضافه دریافت می‌کنند. به عنوان مثال، اگر حقوق ثابت کارمندی ۲۰۰ میلیون ریال باشد، یعنی به ازای هر ساعت یک میلیون ریال دریافت می‌کند. حال اگر چنین فردی ۲۲۰ ساعت کار کند، برای ۲۰۰ ساعت اول ۲۰۰ میلیون ریال و برای ۲۰ ساعت اضافه کار، ساعتی یک و نیم میلیون ریال دریافت می‌کند. در نتیجه کل درآمد این

شخص برای ۲۲۰ ساعت ۲۳۰.۰۰۰.۰۰۰ ریال خواهد بود. فرض کنید این نوع کارمندان جریمه نمی‌شوند و برای کار کمتر از ۲۰۰ ساعت همان حقوق ثابت را دریافت می‌کنند.

(الف - ۴ نمره) با استفاده از چندریختی یک سلسله‌مراتب از کلاس‌ها برای مدل‌سازی کارمندان شرکت تعریف کنید. متد `int hoghoogh(int hours)` برای کارمندان، حقوق آنها را برای `hours` ساعت کار محاسبه می‌کند. فیلدها، سازنده‌ها و متدهای مورد نیاز برای محاسبه حقوق را برای کلاس‌ها تعریف کنید. نیازی به تعریف اعضای دیگر (نام کارمند، ...) نیست.

```
class Employee {
public:
    virtual int hoghoogh(int hours) = 0;
private:
};

class Hourly {
public:
    Hourly(int p) : hourly_payment(p) {}
    int hoghoogh(int hours) {
        return hours * hourly_payment;
    }
private:
    int hourly_payment;
};

class Fixed {
public:
    Fixed(int b) : base_payment(b) {}
    int hoghoogh(int hours) {
        if (hours < 200)
            return base_payment;
        int hourly_payment = base_payment / 200;
        int extra_hours = hours - 200;
        return base_payment + extra_hours * hourly_payment / 2;
    }
private:
    int base_payment;
};
```

(ب - ۲ نمره) کلاسی به نام Company را به طور کامل تعریف کنید که مجموعه‌ای از کارمندان را در خود دارد و دو متد به شرح زیر دارد.

- متد add_employee یک کارمند جدید به آن اضافه می‌کند (پارامتر آن را خودتان تعریف کنید).
- متد int total_payment(int hours) حقوق پرداختی کل شرکت را محاسبه می‌کند اگر تمام کارمندان hours ساعت کار کنند. بدنه این دو متد را بنویسید.

این کلاس را طوری تعریف کنید که با اضافه شدن انواع جدید کارمند تغییری در آن لازم نباشد.

```
class Company {
public:
    void add_employee(Employee* emp) {
        employees.push_back(emp);
    }
    int total_payment(int hours) {
        int total = 0;
        for (Employee* emp : employees)
            total += emp->hoghoogh(hours);
        return total;
    }
private:
    vector<Employee*> employees;
};
```